

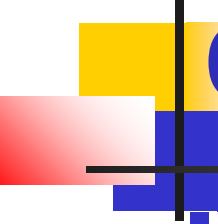
CONVOLUTIONAL NEURAL NETWORK (CNN)

Dr. Nguyen Thi Kim Ngan



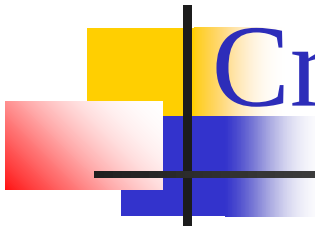
Outline

- Convolutional layer
 - Convolution Operation for Images
 - Padding and Stride
 - Multi-Chanel Input/Output
- Pooling layer
 - Pooling Operation
 - Padding and Stride
 - Multi-Channel Input
- Fully connected layer
- Convolutional Neural Network LeNet



Outline

- **Convolutional layer**
 - Convolution Operation for Images
 - Padding and Stride
 - Multi-Chanel Input/Output
- Pooling layer
 - Pooling Operation
 - Padding and Stride
 - Multi-Channel Input
- Fully connected layer
- Convolutional Neural Network LeNet



Cross-Correlation Operator

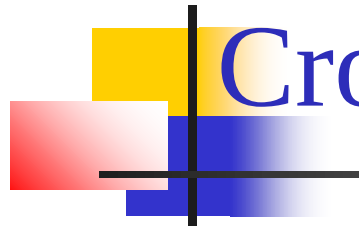
Input				Kernel			Output	
0	1	2	*	0	1	=	19	25
3	4	5		2	3		37	43
6	7	8						

$$0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 = 19$$

$$1 \times 0 + 2 \times 1 + 4 \times 2 + 5 \times 3 = 25$$

$$3 \times 0 + 4 \times 1 + 6 \times 2 + 7 \times 3 = 37$$

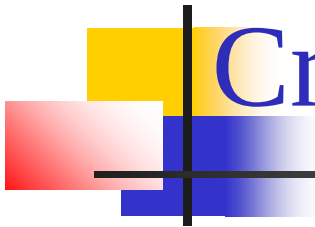
$$4 \times 0 + 5 \times 1 + 7 \times 2 + 8 \times 3 = 43$$



Cross-Correlation Operator

In a convolutional layer

- Input: input array (X), kernel array (K)
- Output: $X \otimes K$ ($\otimes$: cross-correlation operation)



Cross-Correlation Operator

Input		Kernel		Output																	
<table border="1"><tr><td>0</td><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td><td>5</td></tr><tr><td>6</td><td>7</td><td>8</td></tr></table>	0	1	2	3	4	5	6	7	8	*	<table border="1"><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	=	<table border="1"><tr><td>19</td><td>25</td></tr><tr><td>37</td><td>43</td></tr></table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				
3 x 3		2 x 2		$(3-2+1) \times (3-2+1) = 2 \times 2$																	

Input size: $H \times W$

Kernel size: $h \times w$

Output size: $(H - h + 1) \times (W - w + 1)$

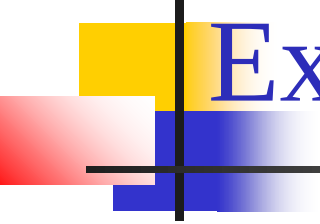
Cross-Correlation Operator

```
from mxnet import autograd, np, npx
from mxnet.gluon import nn
npx.set_np()

# Saved in the d2l package for later use
def corr2d(X, K):
    """Compute 2D cross-correlation."""
    h, w = K.shape
    Y = np.zeros((X.shape[0] - h + 1, X.shape[1] - w + 1))
    for i in range(Y.shape[0]):
        for j in range(Y.shape[1]):
            Y[i, j] = (X[i: i + h, j: j + w] * K).sum()
    return Y
```

Input		Kernel		Output																	
<table><tr><td>0</td><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td><td>5</td></tr><tr><td>6</td><td>7</td><td>8</td></tr></table>	0	1	2	3	4	5	6	7	8	*	<table><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	=	<table><tr><td>19</td><td>25</td></tr><tr><td>37</td><td>43</td></tr></table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				

```
X = np.array([[0, 1, 2], [3, 4, 5], [6, 7, 8]])
K = np.array([[0, 1], [2, 3]])
corr2d(X, K)
```



Example

Implementing a 2D Convolutional Layer

```
class Conv2D(nn.Block):
    def __init__(self, kernel_size, **kwargs):
        super(Conv2D, self).__init__(**kwargs)
        self.weight = self.params.get('weight', shape=kernel_size)
        self.bias = self.params.get('bias', shape=(1,))

    def forward(self, x):
        return corr2d(x, self.weight.data()) + self.bias.data()
```



Example

Object Edge Detection in an Image

- The image has a size of 6×8 pixels. The four middle columns are black (value 0), and the remaining columns are white (value 1)
- The kernel K has a size of 1×2
- Perform the cross-correlation operation between X (the input) and K (the kernel):
 - If two adjacent horizontal elements have the same value, the output will be 0.
 - Otherwise, the output will be nonzero.

1	1	0	0	0	0	1	1
1	1	0	0	0	0	1	1
1	1	0	0	0	0	1	1
1	1	0	0	0	0	1	1
1	1	0	0	0	0	1	1
1	1	0	0	0	0	1	1

1	-1
---	----

0	1	0	0	0	-1	0
0	1	0	0	0	-1	0
0	1	0	0	0	-1	0
0	1	0	0	0	-1	0
0	1	0	0	0	-1	0
0	1	0	0	0	-1	0

```
X = np.ones((6, 8))  
X[:, 2:6] = 0  
X
```

```
K = np.array([[1, -1]])
```

```
Y = corr2d(X, K)  
Y
```



Learning a kernel

```
# Construct a convolutional layer with 1 output channel
# (channels will be introduced in the following section)
# and a kernel array shape of (1, 2)
conv2d = nn.Conv2D(1, kernel_size=(1, 2))
conv2d.initialize()

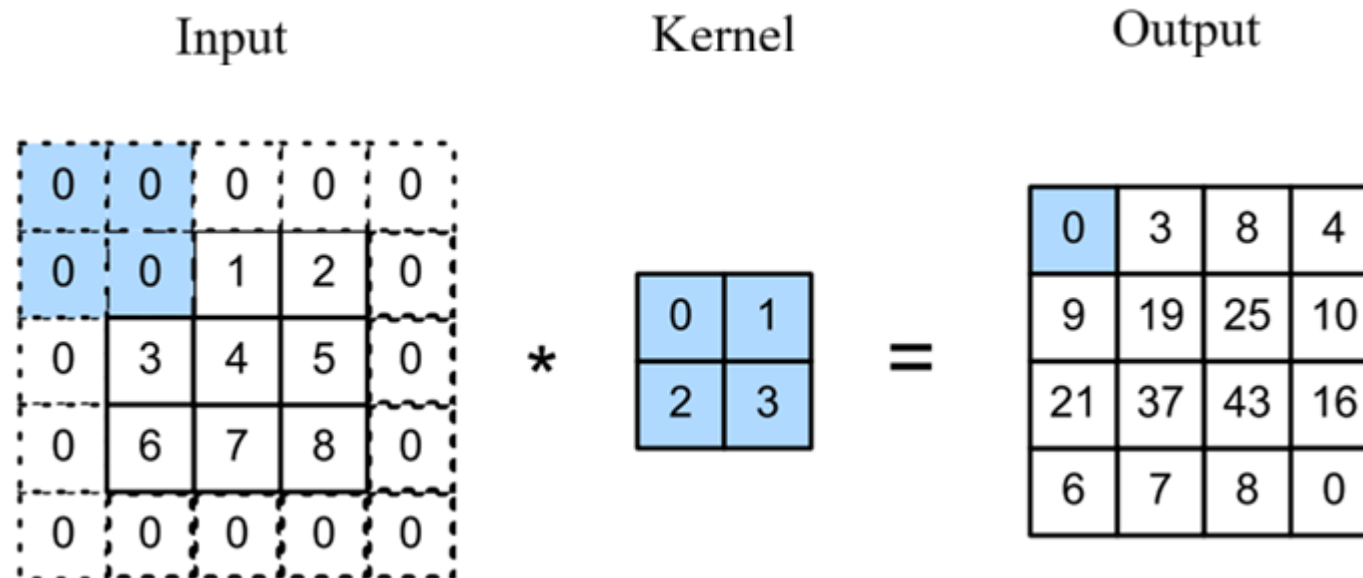
# The two-dimensional convolutional layer uses four-dimensional input and
# output in the format of (example, channel, height, width), where the batch
# size (number of examples in the batch) and the number of channels are both 1
X = X.reshape(1, 1, 6, 8)
Y = Y.reshape(1, 1, 6, 7)

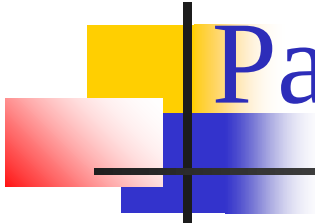
for i in range(10):
    with autograd.record():
        Y_hat = conv2d(X)
        l = (Y_hat - Y) ** 2
    l.backward()
    # For the sake of simplicity, we ignore the bias here
    conv2d.weight.data[:] -= 3e-2 * conv2d.weight.grad()
    if (i + 1) % 2 == 0:
        print('batch %d, loss %.3f' % (i + 1, l.sum()))

conv2d.weight.data().reshape(1, 2)
```

Padding

- Add extra pixels around the borders of the input image to increase its usable size
- Typically, the values of the added pixels are 0





Padding

- If p_h padding rows and p_w padding columns are added, the output size will be:

$$(n_h - k_h + p_h + 1) \times (n_w - k_w + p_w + 1)$$

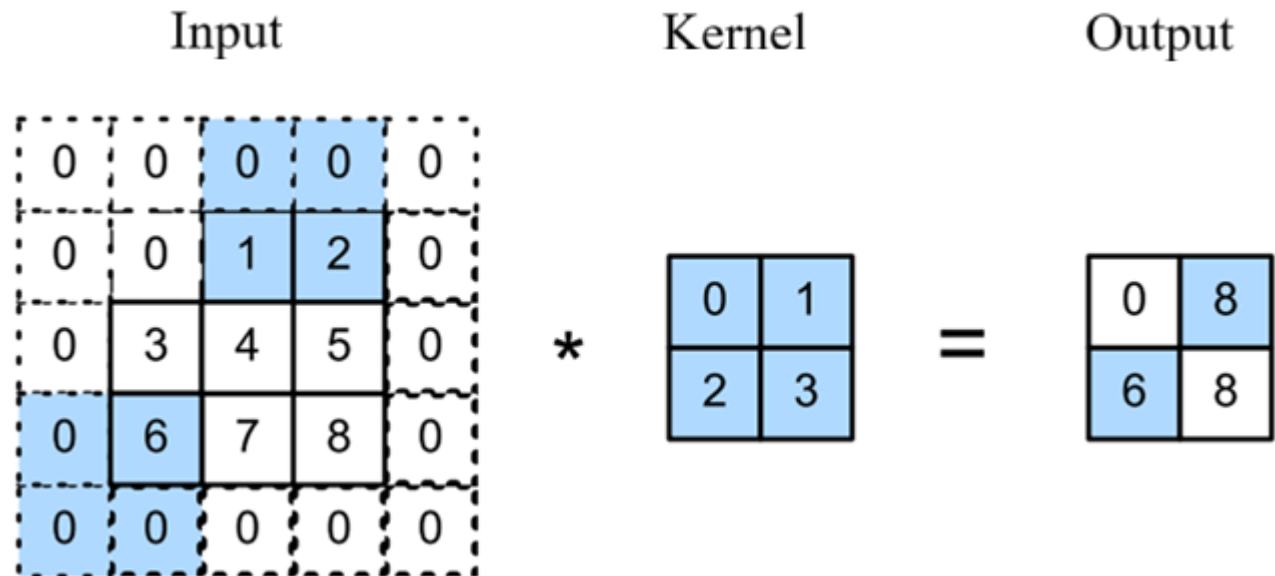
- To make the input and output have the same height and width:

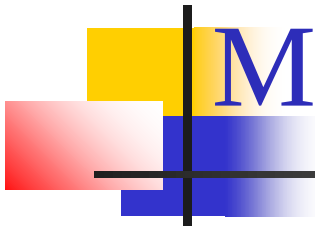
$$p_h = k_h - 1 \text{ and } p_w = k_w - 1$$

Stride

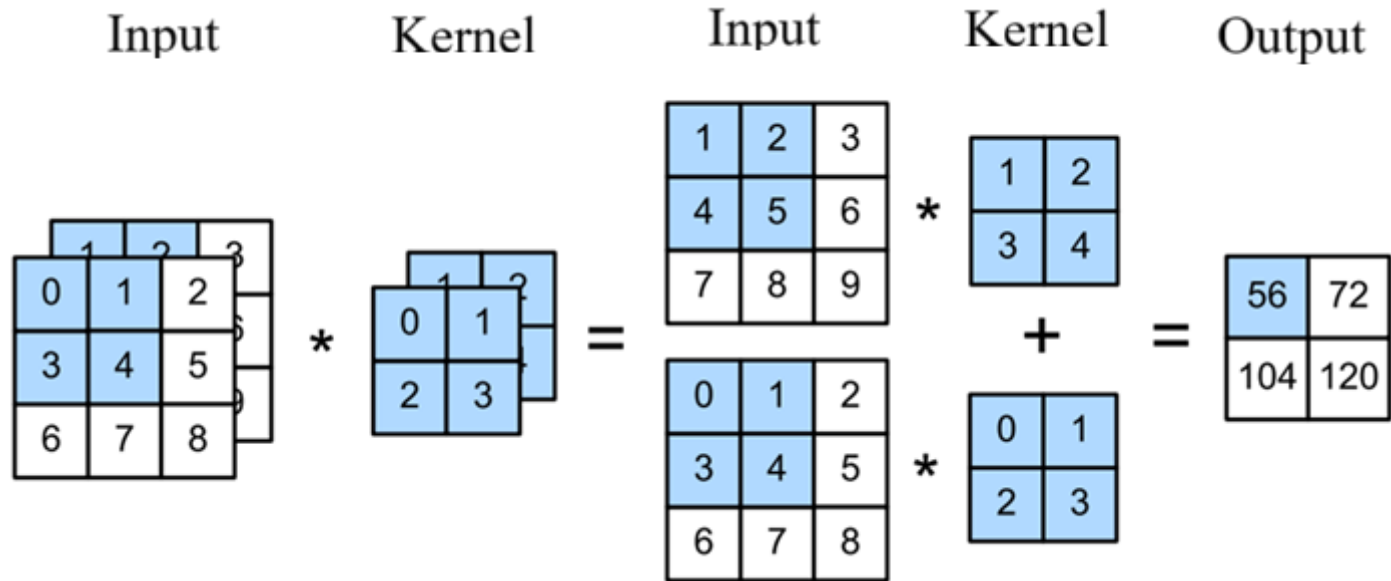
- Stride: the number of rows and columns the kernel moves each step
- For example, a stride of 3 vertically and 2 horizontally
- With vertical stride s_h and horizontal stride s_w , the output size is:

$$\lfloor (n_h - k_h + p_h + s_h) / s_h \rfloor \times \lfloor (n_w - k_w + p_w + s_w) / s_w \rfloor$$



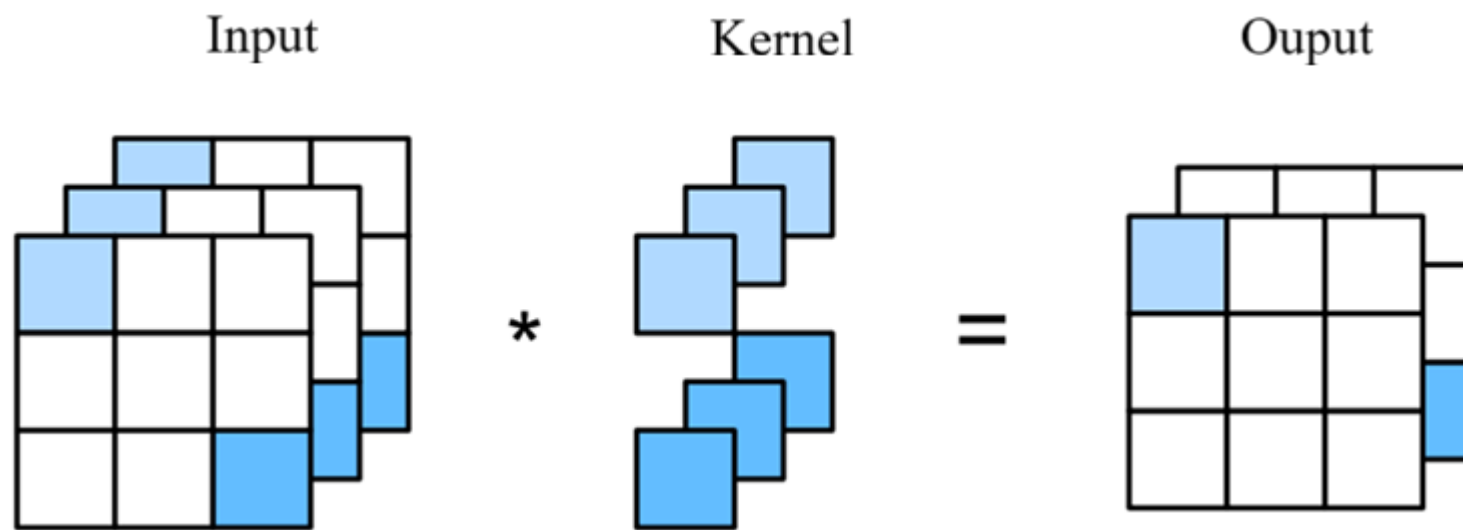


Multi-channel input



$$(1 \times 1 + 2 \times 2 + 4 \times 3 + 5 \times 4) + (0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3) = 56$$

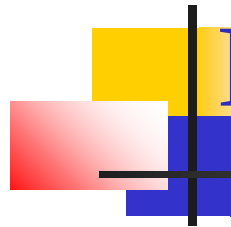
Multi-channel output





Outline

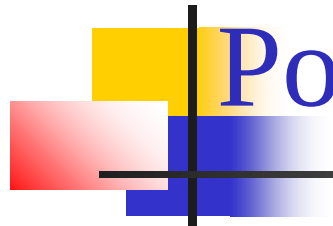
- Convolutional layer
 - Convolution Operation for Images
 - Padding and Stride
 - Multi-Chanel Input/Output
- **Pooling layer**
 - Pooling Operation
 - Padding and Stride
 - Multi-Channel Input
- Fully connected layer
- Convolutional Neural Network LeNet



Pooling layer

Pooling layers are typically used between convolutional layers. Their functions include:

- Reducing the sensitivity of convolutional layers to spatial position
- Reducing the size of the representations



Pooling

- Input: Input array (X), pooling array (K)
- Output: Max pooling (X, K) hoặc Average pooling(X, K)

Max Pooling

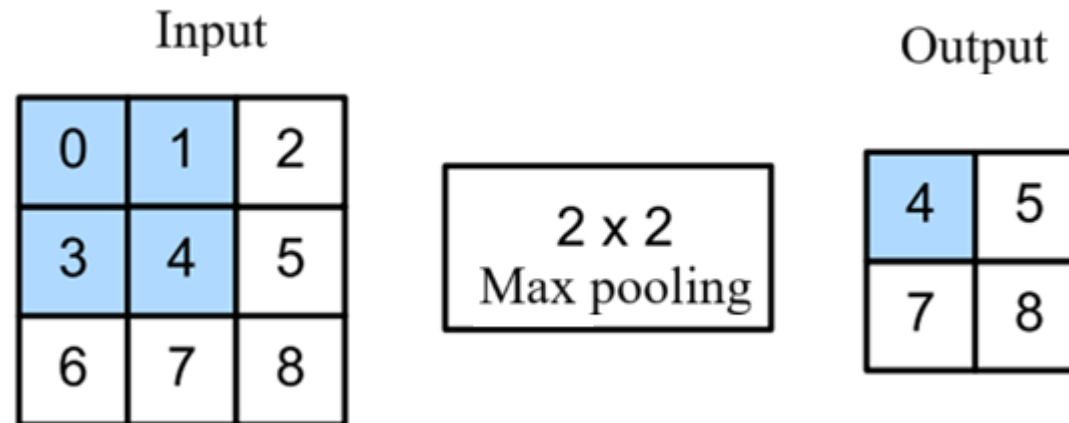
- $p \times q$ pooling layer: computes the maximum value within each $p \times q$ window

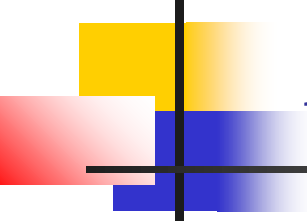
$\max(0, 1, 3, 4) = 4,$

$\max(1, 2, 4, 5) = 5,$

$\max(3, 4, 6, 7) = 7,$

$\max(4, 5, 7, 8) = 8$





Average Pooling

- Average pooling: computes the mean of each $p \times q$ window

$$\text{avg}(0, 1, 3, 4) = 2,$$

$$\text{avg}(1, 2, 4, 5) = 3,$$

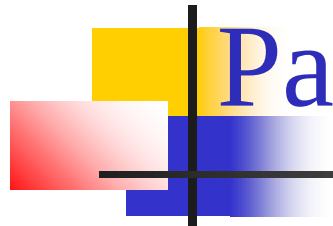
$$\text{avg}(3, 4, 6, 7) = 7,$$

$$\text{avg}(4, 5, 7, 8) = 8$$

0	1	2
3	4	5
6	7	8

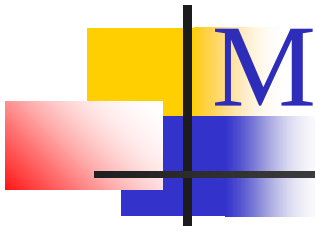
2 x 2
Average
Pooling

2	3
5	6



Padding and Stride

- The padding and stride in a pooling layer are applied in the same way as in a convolutional layer

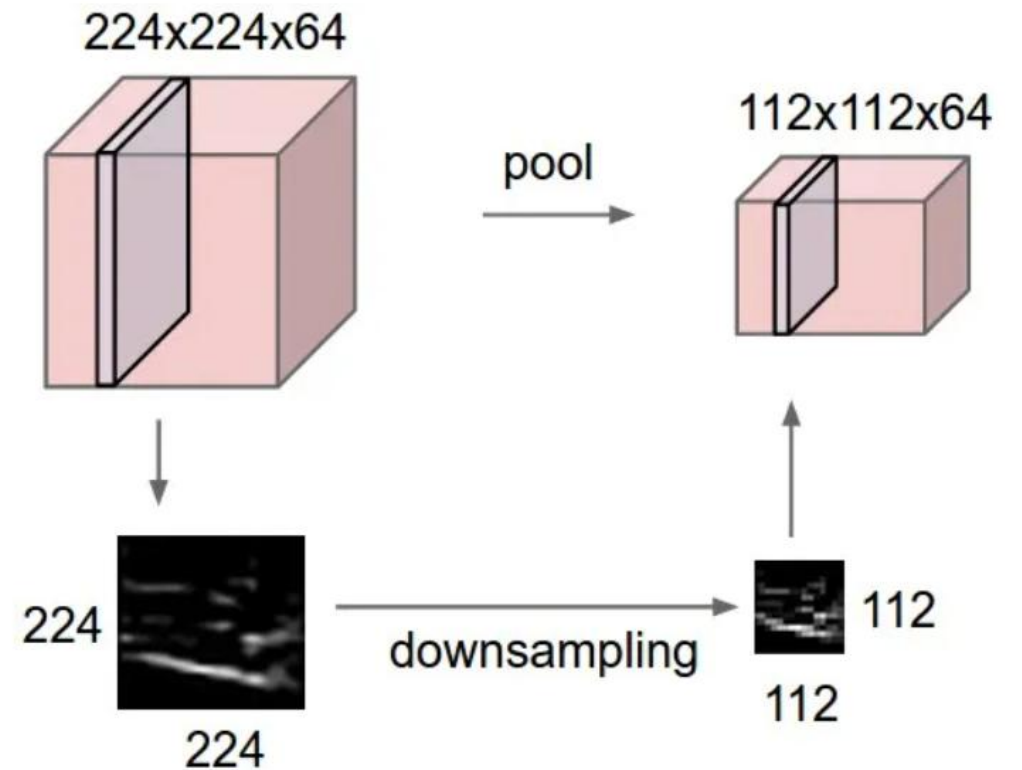


Multi-channel input

- When processing multi-channel input data, the pooling layer is applied to each channel separately rather than summing corresponding elements across channels as in a convolutional layer
- => The number of output channels of the pooling layer will be the same as the number of input channels.

Multi-channel input

- pooling layer: size=(2,2), stride=2, padding=0
- The output width and height of the data are reduced by half, while the depth remains unchanged



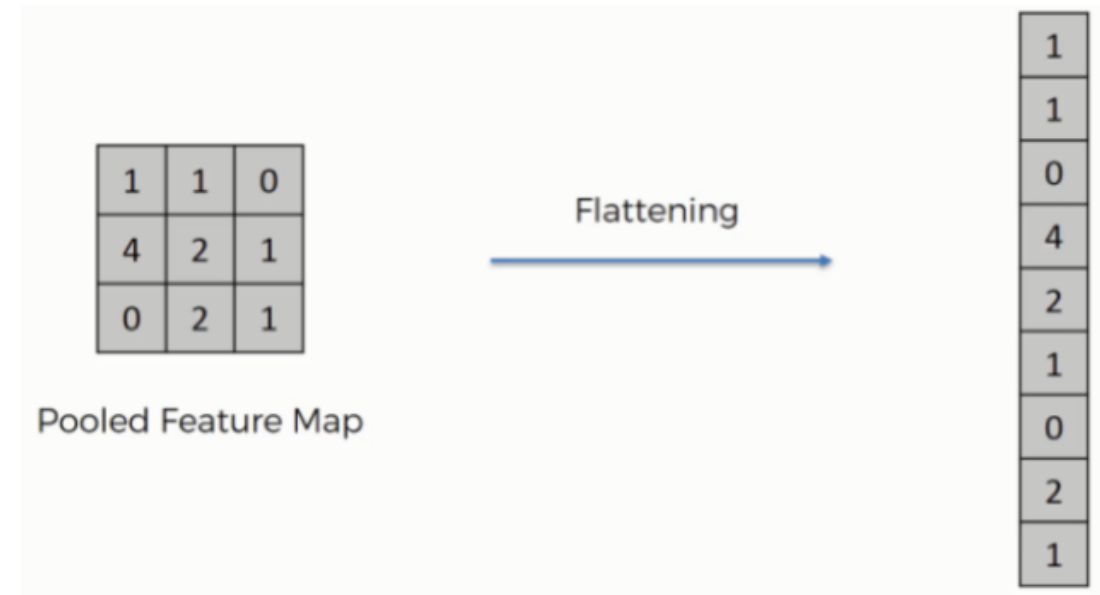


Outline

- Convolutional layer
 - Convolution Operation for Images
 - Padding and Stride
 - Multi-Chanel Input/Output
- Pooling layer
 - Pooling Operation
 - Padding and Stride
 - Multi-Channel Input
- **Fully connected layer**
- Convolutional Neural Network LeNet

Fully connected layer

- After the image passes through multiple convolutional and pooling layers, the output tensor of the last layer (with dimensions $H \times W \times D$) is flattened into a vector (of size $H \times W \times D$)
- Fully connected layers are then used to combine the features of the image to produce the model's output



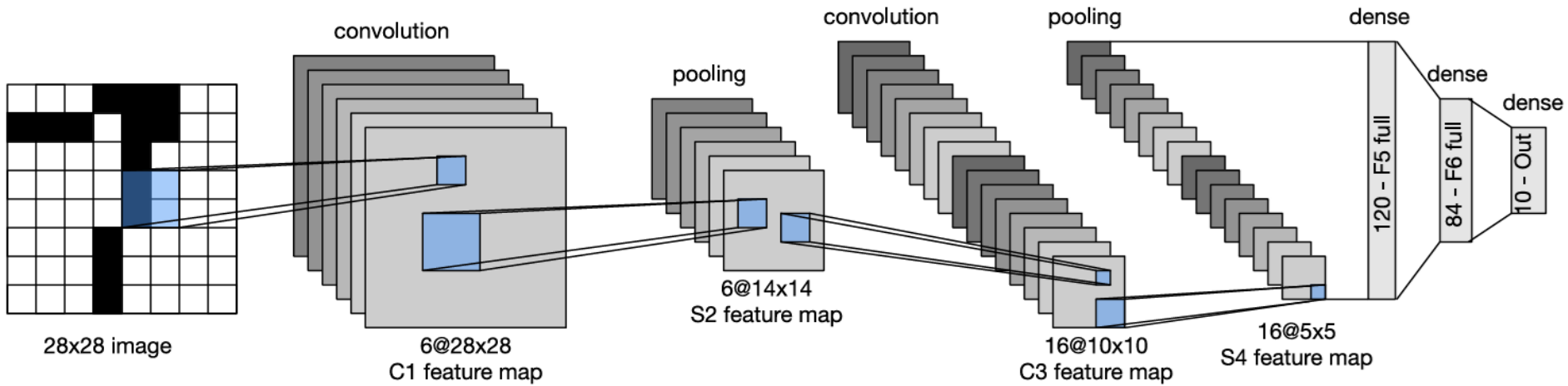


Outline

- Convolutional layer
 - Convolution Operation for Images
 - Padding and Stride
 - Multi-Chanel Input/Output
- Pooling layer
 - Pooling Operation
 - Padding and Stride
 - Multi-Channel Input
- Fully connected layer
- **Convolutional Neural Network LeNet**

Convolutional Neural Network LeNet

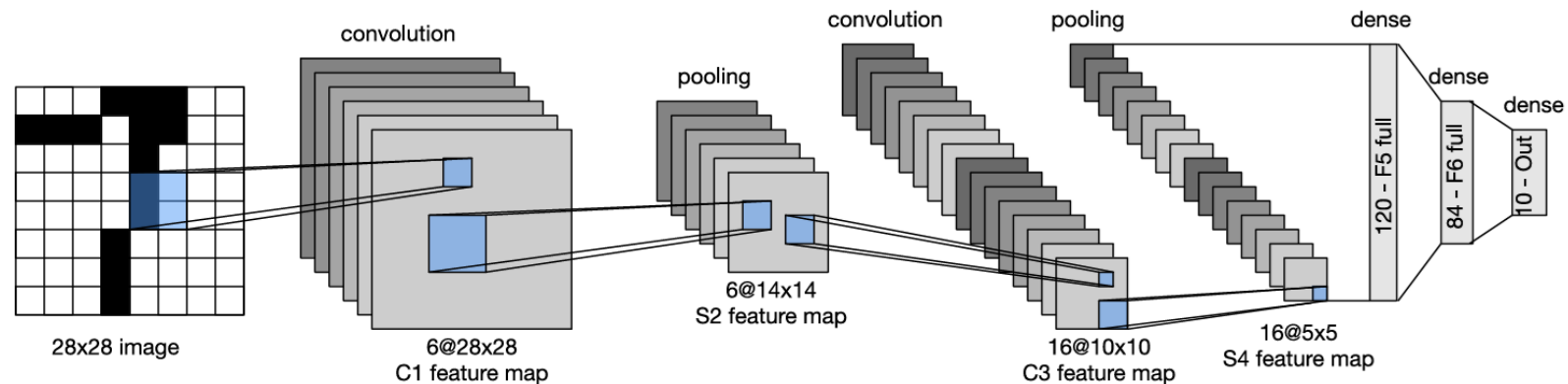
- Convolutional networks were first illustrated by Yann LeCun with the application of handwritten digit recognition in images (LeNet-5) in the 1990s.



Data flow in LeNet. The input is a handwritten digit, the output is a probability over 10 possible outcomes.

Convolutional Neural Network LeNet

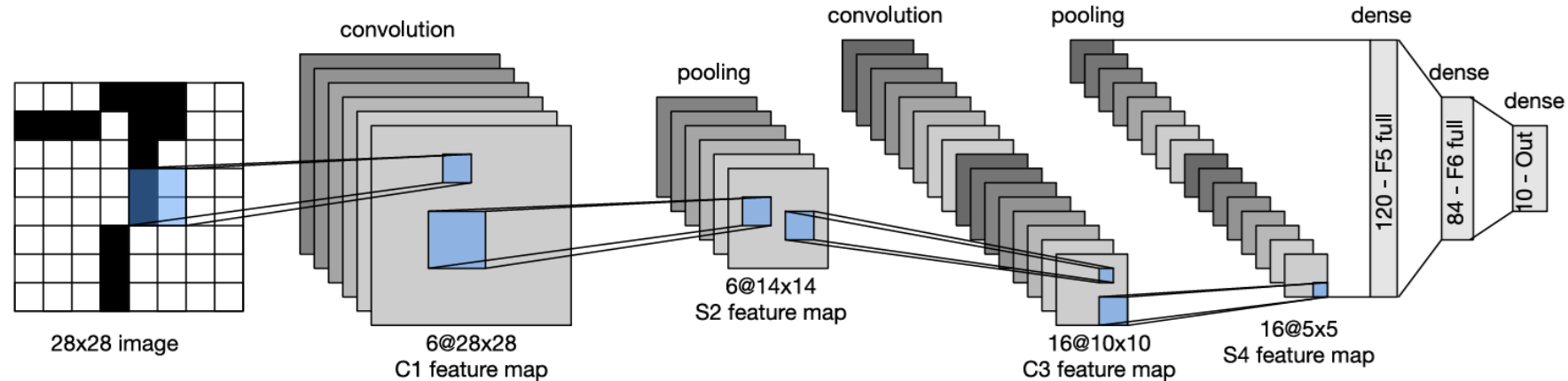
- The convolutional layer is used to detect spatial patterns in images, such as edges and object parts
 - Kernel size = 5×5 , activation function: sigmoid
 - The first convolutional layer has 6 channels, and the second convolutional layer has 16 channels



Data flow in LeNet. The input is a handwritten digit, the output is a probability over 10 possible outcomes.

Convolutional Neural Network LeNet

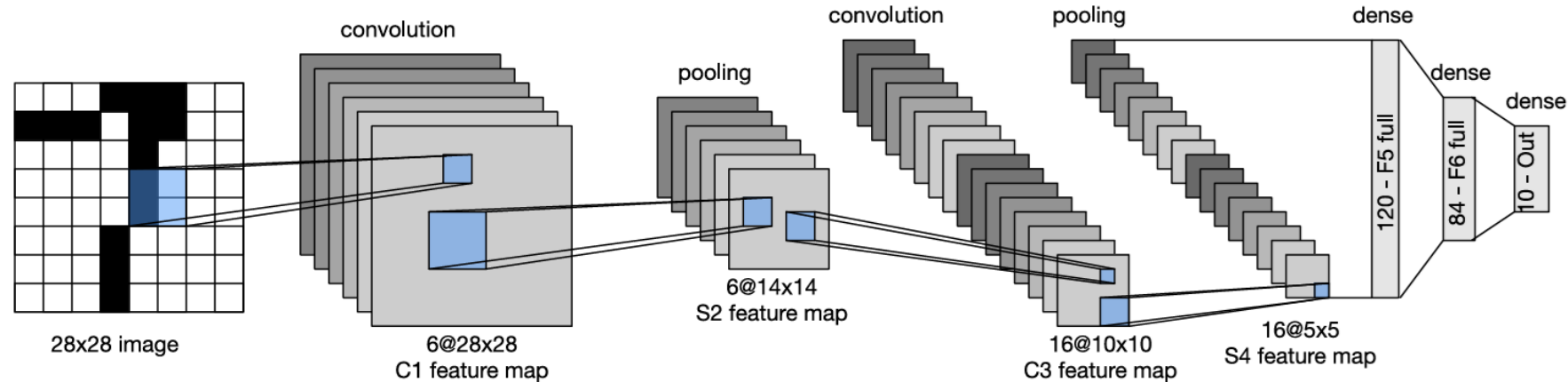
- The following average pooling layer is used to reduce the dimensionality
 - Kernel size = 2×2 , stride = 2



Data flow in LeNet. The input is a handwritten digit, the output is a probability over 10 possible outcomes.

Convolutional Neural Network LeNet

- The fully connected block of LeNet consists of three layers with output sizes of 120, 84, and 10, respectively. (The 10-dimensional output layer corresponds to the number of possible classes — the 10 digits from 0 to 9)

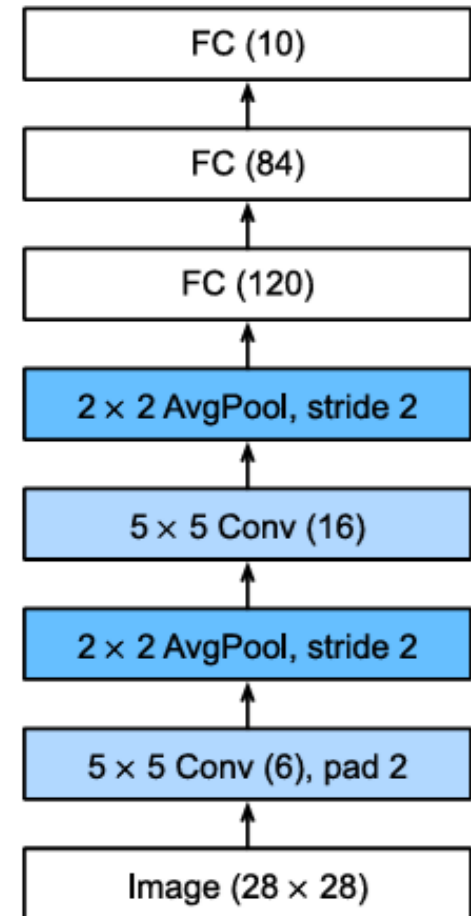


Data flow in LeNet. The input is a handwritten digit, the output is a probability over 10 possible outcomes.

Convolutional Neural Network LeNet

```
from d2l import mxnet as d2l
from mxnet import autograd, gluon, init, np, npx
from mxnet.gluon import nn
npx.set_np()

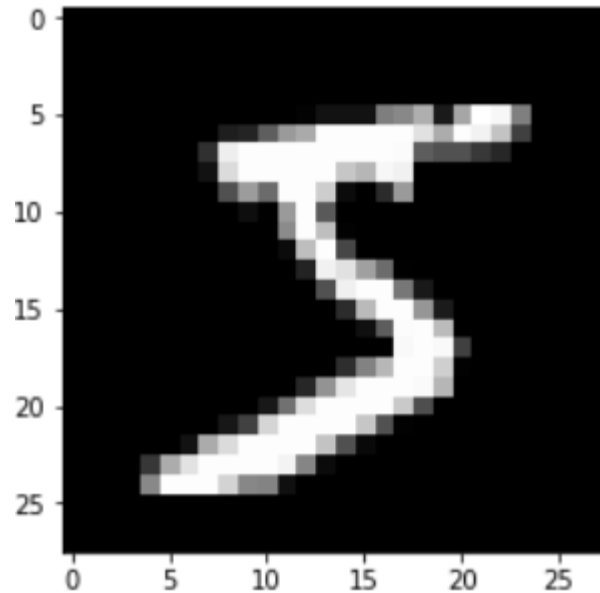
net = nn.Sequential()
net.add(nn.Conv2D(channels=6, kernel_size=5, padding=2, activation='sigmoid'),
        nn.AvgPool2D(pool_size=2, strides=2),
        nn.Conv2D(channels=16, kernel_size=5, activation='sigmoid'),
        nn.AvgPool2D(pool_size=2, strides=2),
        # Dense will transform the input of the shape (batch size, channel,
        # height, width) into the input of the shape (batch size,
        # channel * height * width) automatically by default
        nn.Dense(120, activation='sigmoid'),
        nn.Dense(84, activation='sigmoid'),
        nn.Dense(10))
```



Compressed notation
for LeNet-5.

Problem

- You have a 28×28 grayscale image of a digit from 0 to 9, and you want to predict which digit it is. For example, predict which digit the following image represents



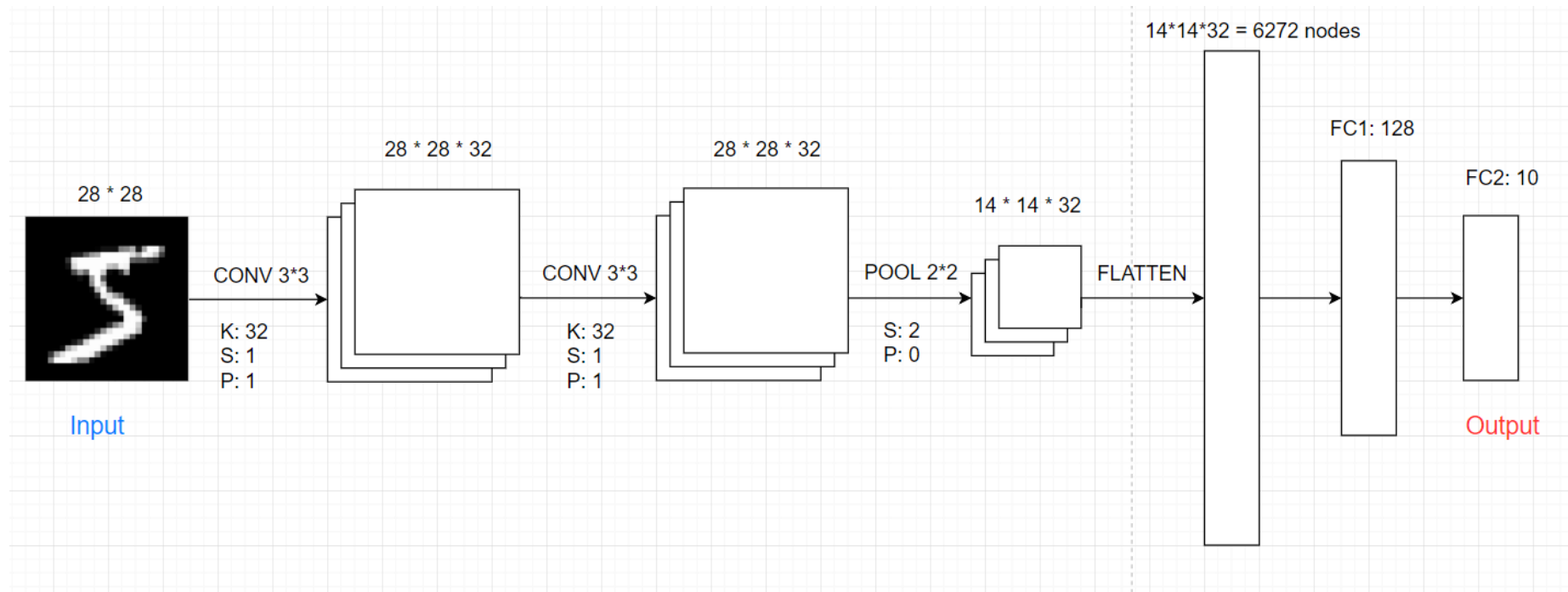


MNIST dataset

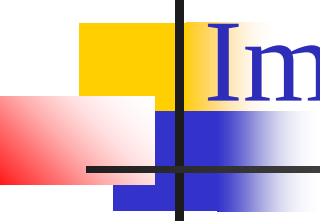
- MNIST is a handwritten digit database, consists: a training set with 60,000 images of handwritten digits and a test set with 10,000 images of handwritten digits
- Preparing the data for the model:
 - Training set: 50.000 samples (images)
 - Validation set: 10.000 samples (images)
 - Test set: 10.000 samples (images)

Building the model

- General model of a CNN: Input image $\rightarrow$ Convolutional layer (Conv) + Pooling layer (Pool) $\rightarrow$ Fully connected layer (FC) $\rightarrow$ Output

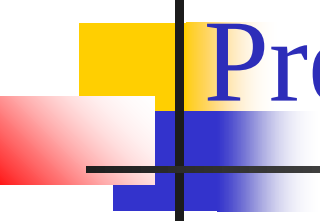


- Input to the model is a 28×28 grayscale image



Import the necessary libraries

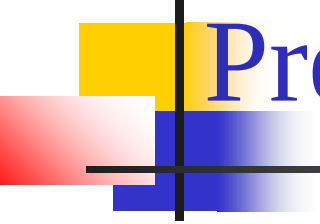
```
import numpy as np
import matplotlib.pyplot as plt
from keras.models import Sequential
from keras.layers import Dense, Dropout, Activation, Flatten
from keras.layers import Conv2D, MaxPooling2D
from keras.utils import np_utils
from keras.datasets import mnist
```



Preparing the datasets

- Load the data from the MNIST dataset, which includes 60,000 training images and 10,000 test images
- Split the training set into 50,000 images for training and 10,000 images for validation

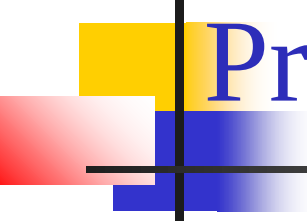
```
(X_train, y_train), (X_test, y_test) = mnist.load_data()  
X_val, y_val = X_train[50000:60000,:], y_train[50000:60000]  
X_train, y_train = X_train[:50000,:], y_train[:50000]  
print(X_train.shape)
```



Preparing the datasets

- The input data for a CNN model is a 4-dimensional tensor (N, W, H, D). In this case, since the images are grayscale, $W = H = 28$, $D = 1$, and N is the number of images per training batch
- Since the image data above has shape (N, 28, 28), i.e., (N, W, H), it needs to be reshaped to $N \times 28 \times 28 \times 1$ to match the shape required by Keras

```
X_train = X_train.reshape(X_train.shape[0], 28, 28, 1)
X_val = X_val.reshape(X_val.shape[0], 28, 28, 1)
X_test = X_test.reshape(X_test.shape[0], 28, 28, 1)
```



Preparing the datasets

- Convert the image labels Y to one-hot encoding
- For example, the digit 5 becomes the vector $[0, 0, 0, 0, 0, 1, 0, 0, 0, 0]$

```
Y_train = np_utils.to_categorical(y_train, 10)
Y_val = np_utils.to_categorical(y_val, 10)
Y_test = np_utils.to_categorical(y_test, 10)
```

Defining model

```
# 5. Định nghĩa model
model = Sequential()

# Thêm Convolutional layer với 32 kernel, kích thước kernel 3*3
# dùng hàm sigmoid làm activation và chỉ rõ input_shape cho layer đầu tiên
model.add(Conv2D(32, (3, 3), activation='sigmoid', input_shape=(28,28,1)))

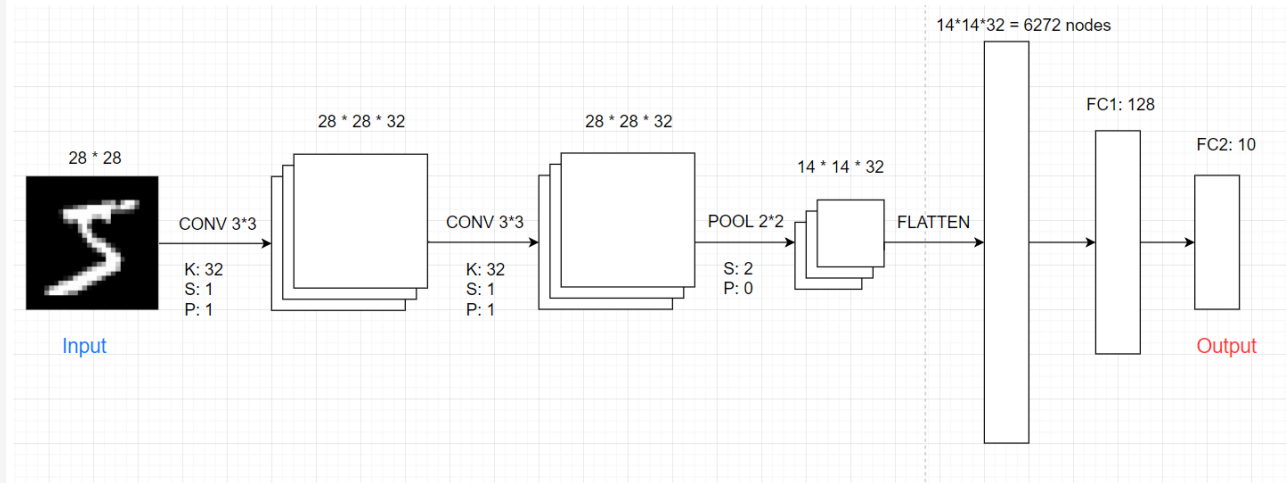
# Thêm Convolutional layer
model.add(Conv2D(32, (3, 3), activation='sigmoid'))

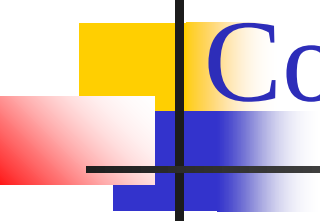
# Thêm Max pooling layer
model.add(MaxPooling2D(pool_size=(2,2)))

# Flatten layer chuyển từ tensor sang vector
model.add(Flatten())

# Thêm Fully Connected layer với 128 nodes và dùng hàm sigmoid
model.add(Dense(128, activation='sigmoid'))

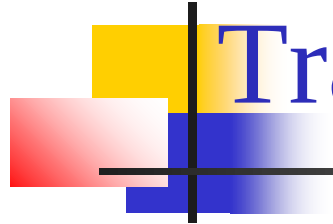
# Output layer với 10 node và dùng softmax function để chuyển sang xác suất.
model.add(Dense(10, activation='softmax'))
```





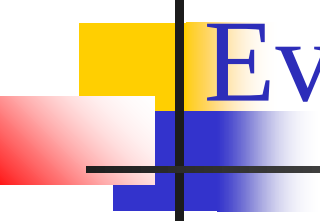
Compile model

```
model.compile(loss='categorical_crossentropy',  
              optimizer='adam',  
              metrics=['accuracy'])
```



Training model

```
H = model.fit(X_train, Y_train, validation_data=(X_val, Y_val),  
              batch_size=32, epochs=10, verbose=1)
```

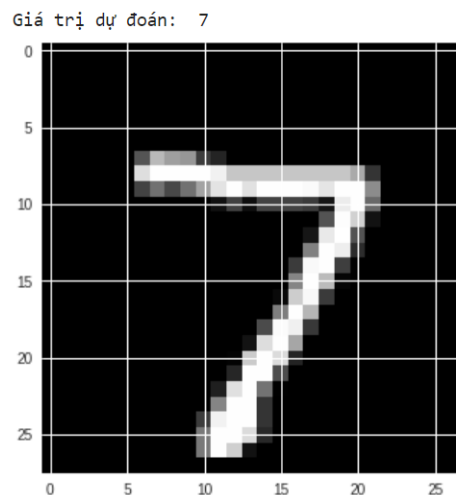


Evaluating the model on the test set

```
score = model.evaluate(X_test, Y_test, verbose=0)  
print(score)
```

Predicting images

```
plt.imshow(X_test[0].reshape(28,28), cmap='gray')  
  
y_predict = model.predict(X_test[0].reshape(1,28,28,1))  
print('Giá trị dự đoán: ', np.argmax(y_predict))
```





Exercise

Build a model for the CIFAR-10 dataset, which includes 50,000 training images and 10,000 test images of 32×32 color images across 10 different classes (airplane, car, ship, bird, dog, cat, horse, ...)

Load the CIFAR 10 data

```
from keras.datasets import cifar10
```

```
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
```